

WYKŁAD 10

Pandas i analiza danych

Kurs języka Python — semestr 2025/2026

Wprowadzenie do programowania II | Informatyka Stosowana

Akademia Nauk Stosowanych w Nowym Sączu

Tematy

1	Pandas — wprowadzenie	Series, DataFrame, tworzenie struktur danych
2	Wczytywanie danych	CSV, Excel, JSON, SQL — <code>read_*</code> ()
3	Eksploracja i typy	<code>head()</code> , <code>info()</code> , <code>describe()</code> , <code>dtypes</code> , <code>NaN</code>
4	Filtrowanie i selekcja	<code>loc</code> , <code>iloc</code> , maski logiczne, sortowanie
5	Operacje na kolumnach	tworzenie, usuwanie, <code>apply()</code> , <code>map()</code>
6	Grupowanie i agregacja	<code>groupby</code> , <code>agg()</code> , <code>pivot_table</code>
7	Łączenie danych	<code>merge()</code> , <code>concat()</code> , <code>join</code> typy
8	Wizualizacja	<code>pandas.plot()</code> , <code>seaborn</code> — szybkie wykresy
9	Praktyczny pipeline	od CSV do raportu — cały workflow

Pandas — wprowadzenie

Struktury danych do analizy tabelarycznej

Czym jest Pandas?

Najważniejsza biblioteka do analizy danych w Pythonie

Cechy biblioteki

- Struktury: Series (1D) i DataFrame (2D)
- Wczytywanie: CSV, Excel, JSON, SQL, Parquet...
- Filtrowanie, grupowanie, agregacja
- Łączenie tabel (merge, concat, join)
- Obsługa braków danych (NaN)
- Integracja z NumPy, Matplotlib, seaborn

Instalacja i import

```
pip install pandas

import pandas as pd
import numpy as np

# Wersja
print(pd.__version__) # np. 2.2.1

# Pandas bazuje na NumPy
# pd.Series → wektorowy (1D)
# pd.DataFrame → tabelaryczny (2D)
```

✓ Konwencja: `import pandas as pd` — tak samo jak `np` dla NumPy, tego aliasu nie zmieniamy.

Series — jednowymiarowa struktura

Jak lista/ndarray, ale z indeksem (etykietą)

Tworzenie Series

```
# Z listy
s = pd.Series([10, 20, 30, 40])
# 0    10
# 1    20
# 2    30
# 3    40

# Z listy + własny indeks
s = pd.Series([10, 20, 30],
              index=['a', 'b', 'c'])

# Ze słownika
s = pd.Series({'Py': 31, 'JS': 25})
```

Operacje na Series

```
s = pd.Series([10, 20, 30, 40],
              index=['a', 'b', 'c', 'd'])

s['b']          # 20
s[1:3]          # b=20, c=30
s[s > 15]       # b=20, c=30, d=40

s.mean()        # 25.0
s.sum()         # 100
s.describe()    # count, mean, std...

# Operacje wektorowe (jak NumPy)
s * 2           # [20, 40, 60, 80]
s + 100         # [110, 120, 130, 140]
np.sqrt(s)      # pierwiastki
```

DataFrame — dwuwymiarowa tabela

Najważniejsza struktura Pandas — tabela z nazwanymi kolumnami i indeksem

```
# Tworzenie z dict (najczęściej)
df = pd.DataFrame({
    'imie':    ['Anna', 'Jan', 'Ewa', 'Piotr'],
    'wiek':    [23, 35, 28, 42],
    'miasto':  ['Kraków', 'Warszawa', 'Gdańsk', 'Kraków'],
    'wyplata': [5500, 8200, 6100, 9500]
})
#   imie  wiek  miasto  wyplata
# 0  Anna   23   Kraków    5500
# 1   Jan   35  Warszawa    8200
# 2   Ewa   28   Gdańsk    6100
# 3  Piotr   42   Kraków    9500

df.shape      # (4, 4)
df.columns    # Index(['imie', 'wiek', 'miasto', 'wyplata'])
df.index      # RangeIndex(start=0, stop=4, step=1)
df.dtypes     # imie: object, wiek: int64, miasto: object, wyplata: int64
len(df)       # 4
```

Inne sposoby tworzenia DataFrame

Z listy słowników

```
dane = [  
    {'imie': 'Anna', 'wiek': 23},  
    {'imie': 'Jan', 'wiek': 35},  
    {'imie': 'Ewa', 'wiek': 28},  
]  
df = pd.DataFrame(dane)
```

Z tablicy NumPy

```
arr = np.array([[1,2,3],[4,5,6]])  
  
df = pd.DataFrame(arr,  
    columns=['a', 'b', 'c'],  
    index=['w1', 'w2'])  
  
#      a  b  c  
# w1   1  2  3  
# w2   4  5  6
```

Dostęp do kolumn i wierszy

```
df['wiek']          # Series – jedna kolumna (notacja słownikowa)  
df.wiek            # to samo (notacja atrybutowa – jeśli nazwa bez spacji)  
df[['imie', 'wiek']] # DataFrame – podzbiór kolumn (podwójne nawiasy!)  
df.iloc[0]         # Series – pierwszy wiersz (po pozycji)  
df.loc[0]           # Series – wiersz o indeksie 0 (po etykiecie)
```

Wczytywanie danych

CSV, Excel, JSON, SQL — `read_*()`

read_csv() — wczytywanie CSV

Najczęściej używana funkcja importu danych

Podstawowe użycie

```
# Najprościej
df = pd.read_csv('dane.csv')

# Z URL
df = pd.read_csv(
    'https://example.com/data.csv')

# Popularne parametry
df = pd.read_csv('dane.csv',
    sep=';',          # separator
    encoding='utf-8', # kodowanie
    header=0,         # wiersz nagłówka
    index_col='id',   # kolumna indeksu
    nrows=1000,       # tylko 1000 wierszy
    usecols=['a','b'], # wybrane kolumny
)
```

Dodatkowe opcje

```
# Daty – automatyczny parsing
df = pd.read_csv('dane.csv',
    parse_dates=['data'],
    dayfirst=True)

# Brakujące dane
df = pd.read_csv('dane.csv',
    na_values=['brak', '-', ''])

# Typy kolumn (oszczędność RAM)
df = pd.read_csv('dane.csv',
    dtype={'wiek': 'int32',
          'kod': 'category'})

# Zapis do CSV
df.to_csv('wynik.csv', index=False)
```

Inne formaty: Excel, JSON, SQL

Excel (.xlsx)

```
# pip install openpyxl
df = pd.read_excel('dane.xlsx',
                  sheet_name='Arkusz1')

# Zapis
df.to_excel('wynik.xlsx',
           index=False, sheet_name='Dane')
```

JSON

```
df = pd.read_json('dane.json')

# Z API (requests + json)
import requests
resp = requests.get(url).json()
df = pd.DataFrame(resp['data'])

df.to_json('wynik.json', orient='records')
```

SQL (sqlite3)

```
import sqlite3
conn = sqlite3.connect('baza.db')

df = pd.read_sql_query(
    "SELECT * FROM tabela", conn)

# Zapis DataFrame do SQL
df.to_sql('tabela', conn,
         if_exists='replace', index=False)
conn.close()
```

Inne formaty

<code>read_parquet()</code>	Parquet — szybki format kolumnowy
<code>read_html()</code>	Tabele z HTML (web scraping)
<code>read_clipboard()</code>	Dane ze schowka (Ctrl+C)
<code>read_feather()</code>	Feather — szybki I/O
<code>read_hdf()</code>	HDF5 — duże dane naukowe

Eksploracja danych

Pierwsze kroki po wczytaniu — poznaj swoje dane

```
df = pd.read_csv('pracownicy.csv')

df.head()          # pierwszych 5 wierszy
df.head(10)        # pierwszych 10 wierszy
df.tail(3)         # ostatnie 3 wiersze
df.sample(5)       # 5 losowych wierszy

df.shape           # (1000, 8) – wiersze x kolumny
df.columns         # nazwy kolumn
df.dtypes          # typy każdej kolumny

df.info()          # podsumowanie: typy, non-null, pamięć
# <class 'pandas.core.frame.DataFrame'>
# RangeIndex: 1000 entries, 0 to 999
# Data columns (total 8 columns):
# #   Column      Non-Null Count  Dtype
# ---  ---
# 0   imie        1000 non-null   object
# 1   wiek        998 non-null    float64   ← brak 2 wartości!

df.describe()      # statystyki: count, mean, std, min, 25%, 50%, 75%, max
df.nunique()        # liczba unikalnych wartości w kolumnach
df['miasto'].value_counts() # ile wystąpień każdej wartości
```

Typy danych i konwersja

object, int64, float64, datetime64, category, bool

Konwersja typów

```
# astype – jawna konwersja
df['wiek'] = df['wiek'].astype(int)
df['kod'] = df['kod'].astype('category')

# pd.to_numeric – bezpieczna konwersja
df['cena'] = pd.to_numeric(
    df['cena'], errors='coerce')
# errors='coerce' → błędne → NaN

# pd.to_datetime
df['data'] = pd.to_datetime(
    df['data'], format='%Y-%m-%d')
```

Typ category — oszczędność RAM

```
# Kolumna z powtarzającymi się wartościami
df['miasto'].memory_usage(deep=True)
# 64128 bajtów (object)

df['miasto'] = df['miasto'].astype(
    'category')
df['miasto'].memory_usage(deep=True)
# 1200 bajtów – 50x mniej!

# Dobre dla: płeć, miasto, status,
# kolor, rozmiar – mało unikalnych
```

Najważniejsze typy Pandas

object	Napisy (str) — domyślny dla tekstu
int64 / float64	Liczby całkowite / zmiennoprzecinkowe
datetime64[ns]	Data i czas — po pd.to_datetime()
category	Kategorie — mało unikalnych wartości, oszczędza pamięć

Brakujące dane — NaN

isna(), dropna(), fillna() — obsługa braków w danych

Wykrywanie braków

```
# Które wartości są brakujące?
df.isna()          # DataFrame bool
df.isna().sum()     # ile NaN w kolumnie
df.isna().sum().sum() # łącznie NaN

# Procent braków
(df.isna().mean() * 100).round(1)

# Które wiersze mają braki?
df[df['wiek'].isna()]
```

Usuwanie i wypełnianie

```
# Usuń wiersze z brakami
df.dropna()          # wszystkie NaN
df.dropna(subset=['wiek']) # tylko z kol.

# Wypełnij braki
df.fillna(0)          # stałą wartością
df['wiek'].fillna(df['wiek'].mean()) # średnią
df.fillna(method='ffill') # wart. poprzednia
df.fillna(method='bfill') # wart. następna

# interpolate – interpolacja
df['temp'].interpolate(method='linear')
```

⚠ dropna() domyślnie nie modyfikuje oryginału! Trzeba albo:
df = df.dropna() albo df.dropna(inplace=True) — to dotyczy większości metod Pandas.

Filtrowanie i selekcja

loc, iloc, maski logiczne, sortowanie

Indeksowanie: loc vs iloc

loc = po etykiecie, iloc = po pozycji (integer location)

loc — etykiety (nazwy)

```
# Pojedynczy wiersz
df.loc[0]          # wiersz o indeksie 0

# Zakres (WŁĄCZNIE!)
df.loc[0:2]        # wiersze 0, 1, 2

# Wiersz + kolumny
df.loc[0, 'imie']   # 'Anna'
df.loc[0:2, ['imie', 'wiek']]

# Filtr + kolumny
df.loc[df['wiek']>30, 'imie']

# Modyfikacja
df.loc[0, 'wiek'] = 25

# Indeks tekstowy
df.set_index('imie', inplace=True)
df.loc['Anna']
```

iloc — pozycje (numery)

```
# Pojedynczy wiersz
df.iloc[0]         # pierwszy wiersz

# Zakres (WYŁĄCZNIE!)
df.iloc[0:2]       # wiersze 0, 1

# Wiersz + kolumna
df.iloc[0, 1]      # wiersz 0, kol 1

# Wiele wierszy + kolumn
df.iloc[0:3, [0, 2]]

# Ostatni wiersz
df.iloc[-1]

# Co n-ty wiersz
df.iloc[:, 2]      # co drugi

# UWAGA: loc[0:2] → 3 wiersze
#         iloc[0:2] → 2 wiersze!
```

Filtrowanie warunkowe

Maski logiczne — główny sposób selekcji wierszy w Pandas

Jeden warunek

```
# Maska logiczna
maska = df['wiek'] > 30
# 0 False, 1 True, 2 False, 3 True

# Zastosowanie maski
df[maska]
df[df['wiek'] > 30]          # to samo

# Porównania z tekstem
df[df['miasto'] == 'Kraków']
df[df['imie'].str.contains('an')]
```

Wiele warunków

```
# AND – operator &
df[(df['wiek']>25) & (df['wiek']<40)]

# OR – operator |
df[(df['miasto']=='Kraków') |
   (df['miasto']=='Gdańsk')]

# NOT – operator ~
df[~(df['wiek'] > 30)]

# isin() – wygodne "OR" na liście
miasta = ['Kraków', 'Gdańsk']
df[df['miasto'].isin(miasta)]
```

query() — elegancka składnia

```
# Zamiast: df[(df['wiek']>25) & (df['miasto']=='Kraków')]
df.query("wiek > 25 and miasto == 'Kraków'")

# Ze zmienną Pythona (prefiks @)
min_wiek = 25
df.query("wiek > @min_wiek")
```

Sortowanie i ranking

sort_values()

```
# Rosnąco (domyślnie)
df.sort_values('wiek')

# Malejąco
df.sort_values('wyplata',
               ascending=False)

# Wiele kolumn
df.sort_values(['miasto', 'wiek'],
               ascending=[True, False])

# Sortowanie indeksu
df.sort_index()
```

Ranking i top-N

```
# Ranking
df['rank'] = df['wyplata'].rank(
    ascending=False)

# Top-5 najwyższe wypłaty
df.nlargest(5, 'wyplata')

# Bottom-3 najniższe
df.nsmallest(3, 'wiek')

# Unikalne wartości
df['miasto'].unique()
df['miasto'].nunique() # ile?
```

✓ `sort_values()` też domyślnie zwraca kopię — pamiętaj o `df = df.sort_values(...)` lub `inplace=True`.

Operacje na kolumnach

Tworzenie, usuwanie, zmiana nazw

```
# Nowa kolumna (wektorowo!)
df['premia'] = df['wyplata'] * 0.1
df['pelne_imie'] = df['imie'] + ' ' + df['nazwisko']

# Warunkowa kolumna
df['senior'] = df['wiek'] >= 40          # bool
df['kategoria'] = np.where(df['wiek']>=30, 'senior', 'junior')

# Usuwanie kolumn
df = df.drop(columns=['premia'])        # zwraca kopię
df.drop(columns=['a', 'b'], inplace=True) # w miejscu

# Zmiana nazw
df = df.rename(columns={'imie': 'imię', 'wiek': 'lat'})
df.columns = ['a', 'b', 'c', 'd']      # wszystkie naraz

# Zmiana kolejności kolumn
df = df[['imie', 'miasto', 'wiek', 'wyplata']]

# Zamiana tekstu w kolumnie
df['miasto'] = df['miasto'].str.lower()
df['miasto'] = df['miasto'].str.strip()    # usun białe znaki
df['miasto'] = df['miasto'].str.replace('kraków', 'KRK')
```

apply(), map(), lambda

Stosowanie funkcji do kolumn i wierszy

apply() — kolumna lub wiersz

```
# Na kolumnie (Series)
df['wiek_kw'] = df['wiek'].apply(
    lambda x: x ** 2)

# Własna funkcja
def kategoria(w):
    if w < 25: return 'junior'
    if w < 40: return 'mid'
    return 'senior'

df['kat'] = df['wiek'].apply(kategoria)
```

map() i replace()

```
# map() — mapowanie ze słownika
mapping = {'Kraków': 'KRK',
           'Warszawa': 'WAW'}
df['kod'] = df['miasto'].map(mapping)
# NaN jeśli brak klucza!

# replace() — podmiana wartości
df['miasto'] = df['miasto'].replace({
    'Kraków': 'KRK',
    'Gdańsk': 'GDA'})

# apply() na całym DataFrame (axis)
df.apply(np.sum, axis=0) # suma kolumn
df.apply(np.sum, axis=1) # suma wierszy
```

⚠ apply() z lambda jest WOLNE — używaj wektorowych operacji Pandas/NumPy, gdy to możliwe:

`df['x'] = df['a'] + df['b']` zamiast `df['x'] = df.apply(lambda r: r['a']+r['b'], axis=1)`

Grupowanie i agregacja

`groupby`, `agg()`, `pivot_table`

groupby() — split-apply-combine

Podziel dane na grupy → zastosuj funkcję → połącz wyniki

Podstawowy groupby

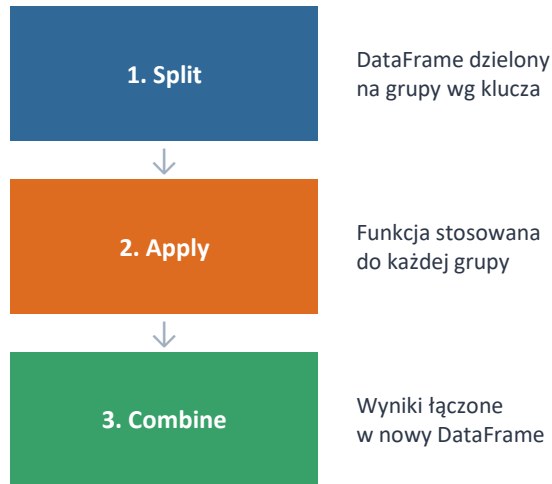
```
# Grupuj po mieście, licz średnią
df.groupby('miasto')['wyplata'].mean()
# Gdańsk      6100
# Kraków      7500
# Warszawa    8200

# Wiele agregacji
df.groupby('miasto')['wyplata'].agg(
    ['mean', 'median', 'count'])

# Grupuj po wielu kolumnach
df.groupby(['miasto', 'stanowisko'])
   ['wyplata'].mean()

# size() vs count()
df.groupby('miasto').size()  # wiersze
df.groupby('miasto').count() # non-NaN
```

Schemat split-apply-combine



Funkcje agregujące i agg()

agg() z nazwą kolumny

```
# Named aggregation (czytelne!)
wynik = df.groupby('miasto').agg(
    sr_wyplata=('wyplata', 'mean'),
    max_wiek=('wiek', 'max'),
    ile=('imie', 'count')
)
#
```

	sr_wyplata	max_wiek	ile
# Gdańsk	6100	28	1
# Kraków	7500	42	2
# Warszawa	8200	35	1

```
# Różne funkcje na różnych kolumnach
df.groupby('miasto').agg(
    {'wyplata': ['mean', 'std'],
     'wiek': 'max'})
```

transform() i filter()

```
# transform – wynik tej samej długości
# (np. "wyplata vs średnia w grupie")
df['sr_grupowa'] = df.groupby(
    'miasto')['wyplata'].transform('mean')

df['odch'] = df['wyplata'] - df['sr_grupowa']

# filter – grupy spełniające warunek
# Tylko miasta z >2 pracownikami
df.groupby('miasto').filter(
    lambda g: len(g) > 2)

# Własna funkcja agregująca
df.groupby('miasto')['wyplata'].agg(
    lambda x: x.max() - x.min())
```

✓ Named aggregation (sr_wyplata=('wyplata','mean')) to najczytelniejszy sposób — używaj go.

pivot_table() i crosstab()

Tabele przestawne — jak w Excelu

pivot_table()

```
pd.pivot_table(df,
               values='wyplata',      # co liczymy
               index='miasto',        # wiersze
               columns='stanowisko',  # kolumny
               aggfunc='mean',        # funkcja
               fill_value=0           # NaN → 0
            )
# stanowisko    dev    mgr    qa
# miasto
# Gdańsk        6100   7500   5800
# Kraków        7200   9500   6000
# Warszawa      8200  10500   7100

# Wiele funkcji
pd.pivot_table(df, values='wyplata',
               index='miasto',
               aggfunc=['mean', 'count'])
```

crosstab() i pivot()

```
# crosstab – tabela kontyngencji
pd.crosstab(df['miasto'],
            df['stanowisko'])
# ile osób w każdej kombinacji

pd.crosstab(df['miasto'],
            df['stanowisko'],
            normalize='index')
# procenty w wierszu

# pivot() – bez agregacji
# (każda kombinacja musi być unikalna)
df.pivot(index='data',
         columns='waluta',
         values='kurs')
```

⚠ pivot() wymaga unikalnych par (index, columns). Jeśli są duplikaty — użyj pivot_table() z aggfunc.

SettingWithCopyWarning

Najczęstszy warning w Pandas — modyfikacja kopii zamiast oryginału

❌ Problem — chained indexing

```
# CHAINED INDEXING — źle!
df[df['wiek'] > 30]['wyplata'] = 9999
# SettingWithCopyWarning!
# Modyfikujesz kopię, nie oryginał!

# Dlaczego?
# df[df['wiek']>30] tworzy kopię
# ['wyplata'] = 9999 modyfikuje tę kopię
# oryginał df NIE jest zmieniony!
```

✅ Rozwiązanie — użyj .loc[]

```
# POPRAWNIE — .loc[] modyfikuje oryg.
df.loc[df['wiek'] > 30, 'wyplata'] = 9999
# ✅ Zmieniony oryginał df

# Pobieranie podtablicy — .copy()
podzb = df[df['wiek'] > 30].copy()
podzb['premia'] = 1000 # OK, na kopii

# Pandas 3.0+ planuje usunąć
# Copy-on-Write (CoW) — wtedy
# każdy wycinek będzie kopią
```

Zasada

Do modyfikacji oryginału zawsze używaj .loc[] / .iloc[]. Do tworzenia podzbioru — zawsze .copy().

Łączenie danych

`merge()`, `concat()`, `join` — SQL-style operacje

merge() — łączenie tabel (SQL-style)

Odpowiednik JOIN w SQL — łączenie po wspólnej kolumnie

```
pracownicy = pd.DataFrame({'id': [1,2,3], 'imie': ['Anna','Jan','Ewa'], 'dep_id': [10,20,10]})
dzialy      = pd.DataFrame({'dep_id': [10,20,30], 'nazwa': ['IT','HR','Finanse']})

# Domyślny inner join
pd.merge(pracownicy, dzialy, on='dep_id')
#   id  imie  dep_id  nazwa
# 0   1  Anna     10     IT
# 1   3   Ewa     10     IT
# 2   2   Jan     20     HR      ← dep_id=30 (Finanse) nie ma dopasowania → pominięty
```

Typy joinów (how=...)

inner	Tylko dopasowane wiersze (domyślny)
left	Wszystko z lewej + dopasowane z prawej (NaN jeśli brak)
right	Wszystko z prawej + dopasowane z lewej
outer	Wszystko z obu tabel (NaN gdzie brak dopasowania)

concat() — sklejanie DataFrame'ów

Sklejanie wierszami (axis=0)

```
df1 = pd.DataFrame({'a':[1,2], 'b':[3,4]})
df2 = pd.DataFrame({'a':[5,6], 'b':[7,8]})

# Wierszami („stos na stos”)
pd.concat([df1, df2])
#   a  b
# 0  1  3 ← z df1
# 1  2  4
# 0  5  7 ← z df2 (indeks!)
# 1  6  8

# Reset indeksu
pd.concat([df1, df2],
          ignore_index=True)
# 0  1  3 ... 3  6  8
```

Sklejanie kolumnami (axis=1)

```
# Kolumnami („obok siebie”)
info = pd.DataFrame({'imie':['A','B']})
dane = pd.DataFrame({'wiek':[23,35]})

pd.concat([info, dane], axis=1)
#   imie  wiek
# 0    A    23
# 1    B    35

# merge vs concat
# merge – łączenie po KLUCZU (jak JOIN)
# concat – sklejanie po OSI (stos/obok)

# Wiele plików CSV naraz
import glob
pliki = glob.glob('dane_*.csv')
df = pd.concat(
    [pd.read_csv(f) for f in pliki],
```

✓ `pd.concat()` z `glob` — świetny sposób na wczytanie wielu CSV do jednego DataFrame naraz.

Wizualizacja

pandas .plot(), seaborn — szybkie i eleganckie wykresy

Wykresy w Pandas — .plot()

Pandas korzysta z Matplotlib pod maską — szybki wykres w jednej linii

Podstawowe typy

```
import matplotlib.pyplot as plt

# Liniowy (domyślny)
df['wyplata'].plot()

# Histogram
df['wiek'].plot.hist(bins=20)

# Słupkowy
df['miasto'].value_counts().plot.bar()

# Kołowy
df['miasto'].value_counts().plot.pie()

# Scatter
df.plot.scatter(x='wiek', y='wyplata')

# Boxplot
df.boxplot(column='wyplata',
            by='miasto')

plt.tight_layout()
plt.show()
```

Opcje .plot()

```
# Parametry wspólne
df.plot(
    kind='bar',           # typ wykresu
    figsize=(10, 6),      # rozmiar
    title='Wypłaty',      # tytuł
    xlabel='Pracownik',   # oś X
    ylabel='PLN',         # oś Y
    color='steelblue',    # kolor
    grid=True,           # siatka
    legend=True,          # legenda
    rot=45                # obrót etykiet
)

# Średnia krocząca (rolling)
df['Close'].rolling(30).mean().plot(
    title='MA 30-dniowa')

# Procentowa zmiana
df['Close'].pct_change().plot.hist(
    bins=50, title='Rozkład zmian')
```

Seaborn — eleganckie wykresy statystyczne

Nakładka na Matplotlib z pięknymi domyślnymi stylami

Instalacja i import

```
pip install seaborn

import seaborn as sns
import matplotlib.pyplot as plt

# Ustaw styl globalnie
sns.set_theme(style='whitegrid')

# Dostępne style:
# darkgrid, whitegrid, dark, white, ticks

# Palety kolorów:
```

Pierwszy wykres

```
# Seaborn przyjmuje DataFrame + nazwy kolumn jako stringi
sns.scatterplot(data=df, x='wiek', y='wyplata', hue='miasto', size='doswiadczenie')
plt.title('Wypłata vs wiek')
plt.tight_layout()
plt.show() # hue = kolor wg kategorii, size = rozmiar wg wartości
```

Seaborn vs Matplotlib

-  Ładniejsze domyślne style
-  Wbudowana obsługa Pandas DataFrame
-  Automatyczne legendy i etykiety
-  Wykresy statystyczne (violin, kde, heatmap)
-  Podział wg kategorii (hue, col, row)
-  Mniej elastyczny niż czysty Matplotlib

Seaborn — przegląd wykresów

Wykresy rozkładowe

```
# Histogram + KDE
sns.histplot(df, x='wiek', kde=True,
             bins=20)

# Sam KDE (gęstość)
sns.kdeplot(df, x='wyplata',
            hue='miasto', fill=True)

# Boxplot
sns.boxplot(data=df, x='miasto',
            y='wyplata', palette='Set2')

# Violin plot
sns.violinplot(data=df, x='miasto',
               y='wyplata')

# Strip / swarm – poszczeg. punkty
sns.stripplot(data=df, x='miasto',
              y='wyplata', jitter=True, alpha=0.5)
```

Wykresy relacyjne i macierzowe

```
# Scatter z regresją
sns.lmplot(data=df, x='wiek',
           y='wyplata', hue='miasto')

# Macierz par zmiennych
sns.pairplot(df, hue='miasto',
             diag_kind='kde')

# Mapa ciepła (korelacja!)
corr = df.select_dtypes('number').corr()
sns.heatmap(corr, annot=True,
            cmap='coolwarm', vmin=-1, vmax=1,
            fmt='.2f')

# Countplot – ile w każdej kategorii
sns.countplot(data=df, x='miasto',
              order=df['miasto'].value_counts()
                .index)
```

Praktyczny pipeline

Od surowych danych do raportu — cały workflow

Krok 1–3: Wczytanie → Eksploracja → Czyszczenie

```
# === KROK 1: Wczytanie ===
df = pd.read_csv('sprzedaz.csv', parse_dates=['data'], encoding='utf-8')

# === KROK 2: Eksploracja ===
print(df.shape)          # (5000, 8)
print(df.dtypes)          # typy kolumn
print(df.isna().sum())    # braki w danych
print(df.describe())      # statystyki
print(df['kategoria'].value_counts())

# === KROK 3: Czyszczenie ===
# Usuń duplikaty
df = df.drop_duplicates()

# Uzupełnij braki
df['cena'] = df['cena'].fillna(df['cena'].median())
df = df.dropna(subset=['produkt']) # usuń wiersze bez produktu

# Konwersja typów
df['ilosc'] = df['ilosc'].astype(int)
df['kategoria'] = df['kategoria'].astype('category')

# Nowa kolumna
df['wartosc'] = df['cena'] * df['ilosc']
df['miesiac'] = df['data'].dt.to_period('M') # okres miesięczny
```

Krok 4–5: Analiza → Wizualizacja

```
# === KROK 4: Analiza (groupby + pivot) ===
# Sprzedaż miesięczna wg kategorii
miesieczna = df.groupby(['miesiac', 'kategoria']).agg(
    suma=('wartosc', 'sum'),
    ile_transakcji=('wartosc', 'count')
).reset_index()

# Top-5 produktów
top5 = df.groupby('produkt')['wartosc'].sum().nlargest(5)

# Pivot: miesiac x kategoria
tabela = pd.pivot_table(df, values='wartosc', index='miesiac',
                        columns='kategoria', aggfunc='sum', fill_value=0)

# === KROK 5: Wizualizacja ===
import seaborn as sns
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 3, figsize=(16, 5))

tabela.plot(kind='bar', stacked=True, ax=axes[0], title='Sprzedaż miesięczna')
top5.plot.barh(ax=axes[1], color='steelblue', title='Top 5 produktów')
sns.boxplot(data=df, x='kategoria', y='wartosc', ax=axes[2])
axes[2].set_title('Rozkład wartości')

fig.tight_layout()
fig.savefig('raport.png', dpi=150)
plt.show()
```

Pandas i analiza danych

1	Series i DataFrame	Series = 1D z indeksem, DataFrame = 2D tabela
2	read_csv() i I/O	wczytywanie CSV, Excel, JSON, SQL — cały ekosystem
3	Eksploracja: head, info, describe	pierwszy krok po załadowaniu danych
4	NaN: isna, dropna, fillna	obsługa brakujących danych — kluczowa umiejętność
5	loc vs iloc	loc = etykiety (włącznie), iloc = pozycje (wyłącznie)
6	groupby + agg	split-apply-combine, named aggregation
7	merge i concat	merge = JOIN po kluczu, concat = sklejanie po osi
8	Seaborn + pandas .plot()	szybka wizualizacja wprost z DataFrame

Następny wykład: Tkinter GUI — okna, widgety, zdarzenia, layout, przykładowe aplikacje